全 JAM 宏觀面試 100 題

100 questions on the JAM protocol

中英文口述題與短答，每輪五題。

點輪次可跳頁。每題上方的章號連到 v0.8.0 原文。

01	JAM 想解決的問題 §1-2 / The problem JAM solves	001-005
02	從規格讀出狀態轉移 §3-4 / Reading the state transition	006-010
03	區塊頭能證明什麼 §5 / What a block header commits to	011-015
04	誰有資格出塊 §6 / Who gets to author blocks	016-020
05	報告怎麼連回歷史 §7 / How reports refer to history	021-025
06	誰可以使用 coretime §8 / Who may use coretime	026-030
07	服務帳戶與資料所有權 §9 / Service accounts and data ownership	031-035
08	出現爭議時，鏈採信什麼 §10 / How a dispute becomes chain evidence	036-040
09	報告上鏈與資料可用 §11 / Reporting and data availability	041-045
10	計算結果何時動到帳本 §12 / When computation changes the ledger	046-050
11	同一區塊裡的輪次與紀錄 §12-13 / Rounds and accounting within a block	051-055
12	Work-package 裡究竟裝什麼 §14 / What a work-package carries	056-060
13	Guarantor 實際承諾什麼 §15 / What a guarantor commits to	061-065
14	資料能不能取回 §16 / Can the data be recovered?	066-070
15	為什麼抽查能建立信任 §17 / Why sampled audits work	071-075
16	最終性與跨鏈證明 §18-19 / Finality and external proofs	076-080
17	效能、設計取捨與範圍 §20-22 / Performance and design trade-offs	081-085
18	同一套 PVM，三種執行情境 App. A-B / One PVM, different invocation contexts	086-090
19	把狀態與資料變成可驗證的承諾 App. C-E / Encoding and Merkle commitments	091-095
20	把各章接成一個可驗證的系統 App. F-I / Putting the protocol together	096-100

JAM 想解決的問題

The problem JAM solves

Q001 §1 §4 §12 §14

一批外部訂單進入 JAM 後，怎麼一路變成鏈上 DEX 的狀態變更？

How does a batch of external orders become a change to a DEX's on-chain state?

> 訂單進入 work-package，guarantors 執行 refinement 並擔保 work-report。報告可用且符合累積條件後，DEX 的 accumulate 可更新帳本。
>
> Orders enter a work-package. Guarantors refine its items and guarantee a report. Once the report is available and eligible, the DEX can apply its results through accumulation.

Q002 §2 §4

如果 DEX 和借貸服務各有自己的鏈，跨鏈使用一筆成交結果時會多出什麼信任問題？

What extra trust question arises when a lending chain uses a trade executed on a separately secured DEX chain?

> 接收方得判斷來源鏈是否正確執行、是否已達最終性。在 JAM 上，兩個服務可以把結果整合進同一套驗證者保護的共享狀態。
>
> The receiver needs evidence that the source chain executed and finalized the trade correctly. On JAM, both services can integrate results into shared state secured by one validator set.

Q003 §1 §4

同一個 DEX 有十批可獨立驗簽的訂單。哪些工作適合平行，哪些決定必須看到共享帳本？

A DEX has ten batches of orders whose signatures can be checked independently. Which work can run in parallel, and which decisions need shared state?

> 不同 core 可平行驗簽並產出結果。涉及同一筆餘額能否扣款，就要在 accumulation 依當時帳本處理。
>
> Different cores can check signatures in parallel. When orders spend the same balance, accumulation must settle them against the applicable ledger state.

Q004 §1 §4 §9

DEX 與預測市場的交易規則不同，JAM 怎麼同時支援兩者？

How can JAM host both a DEX and a prediction market when their transaction rules differ?

> 各 service 用自己的程式定義訂單或下注的意義，並保存自己的狀態。JAM 提供共用的執行、資料可用性與共識機制。
>
> Each service defines its own application rules and keeps its own state. JAM supplies the execution environment, data availability and consensus.

Q005 §2 §4

如果每個驗證者都重跑每一批工作，增加驗證者為什麼很難提升吞吐量？JAM 改變了哪一步？

Why does adding validators do little for throughput if everyone repeats every workload? What does JAM change?

> 全體重跑時，每台機器仍要跟上完整工作量。JAM 把獨立的 refinement 分配到不同 core，再由抽選的驗證者稽核。
>
> With full replication, every machine must keep up with the entire workload. JAM divides refinement across cores and assigns further checks through auditing.

從規格讀出狀態轉移

Reading the state transition

Q006 §3 §4

兩個獨立實作收到同一個區塊。要先對齊哪個輸入，才有理由要求它們算出相同的 state root？

Two independent clients import the same block. What other input must match for them to produce the same state root?

> 兩者要從相同的 prior state 開始。依協定執行同一個區塊，應得到同一個 post-state 與狀態根。
>
> They must start from the same prior state. Applying the specified transition to that state and block must produce the same post-state and root.

Q007 §3 §4 §12

規格在同一頁寫了 prior、帶 dagger 的值和 prime。實作者要怎麼判斷某次檢查該讀哪個版本？

How do you tell whether a check should read the prior, dagger-marked or prime version of a value?

> 看該式引用的版本，沿定義追到產生它的步驟。Prime 通常是這次轉移的結果，dagger 的具體時點由該變數的定義決定。
>
> Follow the version named by the rule back to the step that produces it. A prime usually denotes the transition result. A dagger's precise point is defined for that variable.

Q008 §3 §12

把一串待處理的 reports 改存進無序集合，可能在哪裡改變鏈上結果？

How could storing an ordered sequence of reports in an unordered set change the chain result?

> 有 gas 限制時，順序決定哪段前綴能執行。若兩個節點以不同順序取出報告，就可能更新出不同的狀態。
>
> Under a gas limit, order determines which prefix can run. Nodes that iterate over reports in different orders could produce different states.

Q009 §3 §4

兩個開發團隊把區塊驗證步驟排成不同順序，什麼情況下兩者仍可能都正確？

When can two clients evaluate block-transition components in different orders and both be correct?

> 只要都滿足資料依賴，每一步取到規格指定版本的輸入，互不依賴的項目就能交換順序或平行計算。
>
> Independent components can be reordered or parallelized when each receives the exact input versions specified by the dependency graph.

Q010 §4 §5

某個區塊沒有使用者訂單。它仍可能讓哪些協定工作往前走？

What protocol work can a block advance even if it carries no user orders?

> 區塊推進時槽與近期歷史，也可納入 tickets、assurances 或 disputes 等資料，驅動相應的鏈上轉移。
>
> It advances the timeslot and recent history. Extrinsics such as tickets, assurances or disputes can also advance their respective protocol state.

區塊頭能證明什麼

What a block header commits to

Q011 §5

同步節點同時檢查 parent hash 和 prior state root。它們各自固定了什麼？

What do the parent hash and prior state root each establish for a syncing node?

> Parent hash 指定要接在哪個父區塊。Prior state root 承諾執行本區塊的起始狀態，也就是父區塊執行後的狀態。
>
> The parent hash identifies the preceding block. The prior state root commits to this block's starting state, which is its parent's post-state.

Q012 §5 §7

JAM 為什麼把前一個狀態根放進 header？這對出塊流程有什麼好處？

Why does a JAM header carry the prior state root, and how does that help block production?

> 當前 header 可以引用已知的父狀態，把本區塊執行與計算新狀態根安排成管線。新根可由子區塊承諾。
>
> Referencing the parent state lets block computation and Merklization be pipelined. A child header can commit to the newly computed root.

Q013 §5

有人只給你 header 和一份 work-report。要怎麼驗證這份 report 確實被該區塊收錄？

Given a header and a work-report, how can you verify that the block included that report?

> 取得相應的 inclusion proof，依協定的編碼與 Merkle 規則重建承諾，再比對 header 裡的 extrinsic hash。
>
> Obtain the inclusion proof, reconstruct the commitment using the protocol's encoding and Merkle rules, and compare it with the header's extrinsic hash.

Q014 §5 §6

父區塊後跳過好幾個 slot 才出現下一個區塊。節點應怎麼判斷它的時間是否合法？

A child block arrives after several empty slots. How should a node evaluate its timeslot?

> 子區塊的 slot 必須比父區塊晚，且已到本地時鐘允許驗證的時間。中間可以空 slot，因此高度不能代替時槽。
>
> Its slot must be later than its parent's and admissible under the current clock. Slots may be empty, so block height cannot stand in for the slot index.

Q015 §5 §6 §10

其他節點如何從 header 得知驗證者換屆、下期出塊票券或新違規者等關鍵變動？

How does a header communicate epoch changes, winning tickets and newly identified offenders?

> 協定在指定條件下寫入對應的 markers。接收方依狀態轉移核對這些公告，下一期排程與新違規者因此有共同依據。
>
> The header carries the corresponding markers when their conditions apply. Nodes check them against the transition, giving them a shared record of schedules and new offenders.

誰有資格出塊

Who gets to author blocks

Q016 §6 §19

Safrole 已經指定某個 slot 的作者。為什麼之後還要 GRANDPA？

If Safrole assigns an author to a slot, what work remains for GRANDPA?

> Safrole 管理鏈如何延長，仍可能出現競爭分支。GRANDPA 用驗證者投票確定一段共同歷史。
>
> Safrole governs chain extension, where competing branches can still arise. GRANDPA uses validator votes to finalize a common history.

Q017 §6 App. G

為什麼可以先公布中選票券，又不必提前公開每張票屬於哪個驗證者？

How can winning tickets be public while the identities of their holders remain hidden in advance?

> Ring VRF 讓驗證者證明自己是合格集合的一員並產生可驗證的票券。持票者到指定 slot 才用相應證明封塊，較難被預先鎖定。
>
> A ring VRF proves eligible membership and produces a verifiable ticket without revealing which member owns it. The holder later proves entitlement when sealing the block.

Q018 §6

換 epoch 時，如果票券競賽未能產生可用的完整排程，鏈靠什麼繼續出塊？

How can the chain continue if the ticket contest fails to yield a usable schedule for the next epoch?

> Safrole 依規則從驗證者金鑰與既有熵產生 fallback 排程。各節點可獨立算出同一組有資格封塊的金鑰。
>
> Safrole derives a fallback schedule from validator keys and recorded entropy. Nodes independently recover the same eligible sealing keys.

Q019 §6 §12

一個服務今天提出下期的驗證者名單，為什麼不能立刻用新名單驗今天的區塊？

Why would a newly proposed validator list be inappropriate for verifying today's blocks?

> 金鑰有 staging、pending、active 等階段，並在 epoch 邊界依規則輪替。驗當期區塊要看規格為它指定的 active set。
>
> Keys pass through staging, pending and active sets at epoch transitions. A block must be checked against the active set prescribed for that block.

Q020 §6 §11

分配驗證者任務需要隨機性。不同節點怎麼拿到相同的隨機來源？

How do nodes obtain a common source of randomness for validator assignments?

> 區塊的 VRF 輸出按規則混入 entropy accumulator。協定保留歷史 epoch 的熵，指定任務使用其中哪一份。
>
> Block VRF outputs feed a shared entropy accumulator. The protocol retains epoch snapshots and specifies which snapshot seeds each assignment.

報告怎麼連回歷史

How reports refer to history

Q021 §7 §11

一份新 report 聲稱它參照某個近期區塊。節點用什麼來核對這個 anchor？

How does a node check a new report's claim about its recent anchor block?

> 比對近期歷史中保存的 header hash、slot、狀態根與累積輸出承諾。這讓引用過期或捏造近期背景的報告無法通過該檢查。
>
> The node matches the header hash, slot, state root and accumulation-output commitment against recent history. A stale or fabricated recent context fails this check.

Q022 §7 §11 §14

後一份工作聲稱「套件 P 產生的 segments root 是 S」。節點為什麼能核對這組對應？

How can a node verify a claim that package P produced segments root S?

> 近期歷史保留已報告的 package hash 到 segments root 的對應，當前區塊的報告也可提供依據。節點用這些紀錄核對引用。
>
> Recent history records package hashes and their reported segment roots. Reports in the current block can also establish the mapping used for this check.

Q023 §7 §9 §11

如果兩個節點現在看到的服務資料已不同，日後重跑舊工作時要讀哪個版本？

If a service's data has changed, which version should a validator use when reproducing earlier work?

> 用該工作的 lookup anchor 固定歷史視角，依那個時點的規則查程式或 preimages。節點因此能重現原本可見的輸入。
>
> The work's lookup anchor fixes the historical view for code and preimage lookups. A validator can reproduce the inputs that were visible at that point.

Q024 §7 §12

在 recent history 找到套件 P 的 hash，就能斷定它已被累積嗎？

Does finding a package hash in recent history prove that the package was accumulated?

> 它能證明 P 曾被報告。要確認已經累積，還要查 accumulation 的完成紀錄，兩者追蹤的階段不同。
>
> It establishes that P was reported. Accumulated-package history records which packages actually reached accumulation.

Q025 §7 §12 §18

外部系統只想驗證某個服務發布的累積輸出。JAM 如何把這類輸出留成可供證明的歷史？

How does JAM preserve a provable history of outputs published by services during accumulation?

> 每個區塊把服務識別碼與輸出雜湊組成序列，對它取根，再把根追加到 accumulation output log。之後可針對那筆輸出建立包含證明。
>
> Each block commits to its sequence of service IDs and output hashes. The block-level root is appended to the accumulation output log, supporting later inclusion proofs.

誰可以使用 coretime

Who may use coretime

Q026 §4 §8

團隊今天就想買下週的 coretime，當時還不知道使用者會送哪些訂單。為什麼辦得到？

How can a team buy next week's coretime before it knows which orders users will submit?

> 可以先分配 core 的執行資源，把挑選具體工作的規則交給 authorizer。之後提交的 work-package 再按那些規則接受檢查。
>
> Coretime can be allocated in advance. An authorizer then applies the agreed rules to a specific work-package when it is submitted.

Q027 §8 §15

一份工作聲稱自己有權使用某個 core。誰提供授權證據，誰判斷，成功後會留下什麼？

Who supplies evidence that a package may use a core, who evaluates it, and what does success produce?

> 套件帶入 token，authorizer 程式執行授權判斷。成功時產生 trace，供後續報告描述這次授權。
>
> The package supplies a token. The authorizer evaluates the request and, on success, returns a trace that is carried in the report.

Q028 §8 §11

執行 authorizer 得到成功結果後，為什麼鏈上還要檢查該 core 的 authorizer pool？

Why check the core's authorizer pool on-chain after the authorizer has approved the package?

> 程式成功表示工作符合該 authorizer 的規則。鏈上還要確認這個 authorizer 的 hash 當時確實獲准代表該 core 授權。
>
> Execution shows that the package satisfied that authorizer's rules. Pool membership shows that this core actually accepted that authorizer at the relevant time.

Q029 §8

一個合法 authorizer 寫在 pool 裡，怎麼避免同一筆授權一直被重用，又讓長期未用的授權退出？

How does a pool consume an authorization that was used, while also retiring stale entries?

> 有 guarantee 時，先移除它用掉的最早匹配項，再從 queue 補值。補值後若超過容量，保留最新的項目。
>
> A guarantee consumes the oldest matching pool entry. The transition appends the scheduled queue entry and keeps the newest entries up to the pool limit.

Q030 §8 §9 §12

某個 core 想改未來接受工作的政策。誰能改排程，這項變動何時進入 pool 計算？

Who can change a core's future authorization schedule, and when does the pool use it?

> 該 core 的特權 assigner 可在 accumulation 改 authorizer queue。區塊計算新 pool 時，按目前 slot 從更新後的 queue 取值。
>
> The core's privileged assigner may change the queue during accumulation. The pool update selects the current slot's entry from that updated queue.

服務帳戶與資料所有權

Service accounts and data ownership

Q031 §4 §9

DEX 的下一批工作換到另一個 core。原本的訂單簿和帳戶資料存在哪裡？

A DEX moves its next workload to another core. Where do its order book and account data live?

> 鏈上帳本跟著 service account 的儲存空間。Core 提供執行工作所需的資源，換 core 之後仍可由同一服務更新同一份帳本。
>
> The ledger lives in the service account's storage. A core supplies execution capacity, so a later workload can run on another core and still feed the same service.

Q032 §9

服務想永久多存一百萬筆很小的資料，JAM 如何讓狀態空間的占用反映在成本上？

How does JAM make a service account for the cost of storing a million small entries?

> 服務帳戶的門檻餘額取決於儲存項目數和總位元組數，並計入適用的儲存抵免。拆成許多小項目也有成本。
>
> The account's threshold balance reflects both entry count and byte footprint, subject to applicable storage credit. Many small entries therefore carry a storage cost.

Q033 §9 §14

服務剛升級程式。要重跑一份升級前建立的 work-item，怎麼知道該執行哪份程式碼？

A service has upgraded. How can a validator identify the code for a work-item created before the upgrade?

> Work-item 指定 service ID 和 code hash。驗證者依 lookup anchor 的歷史視角取得該 hash 的程式原文。
>
> The work-item names the service and code hash. The validator looks up the matching code preimage using the historical view fixed by the lookup anchor.

Q034 §9 §12

服務想按需取得一份大型文件。從提出 preimage 請求到原文進入帳戶，中間要符合什麼條件？

How does a service request a large document by hash and later receive the bytes through a block preimage extrinsic?

> 服務先登記 hash 與長度。以區塊 extrinsic 提供時，請求在 prior state 必須仍待供應，內容吻合且在整合時仍需要，才寫入帳戶。
>
> The service requests a hash and length. For a block provision, the request must be pending in prior state. Matching bytes are stored if the request remains relevant at integration.

Q035 §9 §12

DEX 使用者轉了 500 枚應用代幣，跟 JAM 服務帳戶之間轉移 500 單位餘額，各改到哪份帳？

Which ledger changes for a transfer of 500 DEX tokens versus a transfer of 500 units between JAM services?

> 前者依 DEX 的規則更新應用帳本，例如 service storage 裡的餘額。後者移動協定管理的 service-account balance。
>
> The DEX applies its token rules to its application ledger, for example balances in service storage. A protocol transfer moves balance between service accounts.

出現爭議時，鏈採信什麼

How a dispute becomes chain evidence

Q036 §10 §17

一名稽核者認為 report 算錯了。它簽署的 judgment 和可以納入區塊的 verdict 有何差別？

How does one auditor's signed judgment differ from a verdict that a block can include?

> Judgment 是單一驗證者對報告的判斷。Verdict 彙整符合規則的驗證者簽名及票數，讓所有節點可重算裁決。
>
> A judgment is one validator's signed assessment. A verdict collects the required judgments so all nodes can verify the signers, votes and outcome.

Q037 §10

同一份爭議報告隔天又被提交。保留 good、bad、wonky 這些歷史紀錄有什麼作用？

Why retain good, bad and wonky report histories when a disputed report might reappear later?

> 節點可以辨認過去已判定有效、無效或無法確定的報告，依既有裁決拒絕重複爭議或不合格的再提交。
>
> Nodes can recognize a report already judged valid, invalid or indeterminate. The recorded outcome supports rejecting repeat verdicts and inadmissible resubmissions.

Q038 §10 §11

一份 report 還在某個 core 上等 assurance，此時鏈上出現 bad 或 wonky verdict。排程要怎麼處理？

A report is still awaiting assurance when a bad or wonky verdict is recorded. What happens to the assignment?

> 該 core 上與報告相符的 pending availability assignment 會被清除，這份報告不會循這個指派進入可用報告序列。
>
> The matching pending availability assignment is cleared. That assignment can no longer move the report into the sequence of newly available reports.

Q039 §10

一個人簽了錯誤的 guarantee，另一個人簽了與裁決相反的 judgment。鏈上如何區分責任？

How does the chain distinguish guaranteeing an invalid report from issuing a judgment that contradicts the verdict?

> Culprit 證據指出誰擔保了無效報告。Fault 證據指出誰簽了與有效性裁決相反的 judgment。兩者都要能驗簽並對應到報告。
>
> Culprit evidence identifies a guarantor of an invalid report. Fault evidence identifies a signer of a contradicting judgment. Both require signatures tied to the report.

Q040 §10 §13

已查到違規驗證者的金鑰。從認定違規到實際處分，JAM 和上層制度各負責什麼？

After an offending validator is identified, what does JAM record and who can impose economic penalties?

> JAM 保存可驗證的裁決與違規金鑰，讓節點對事件有共同紀錄。上層的驗證者選擇或 staking 系統可據此執行懲罰。
>
> JAM records verifiable verdicts and offending keys. A higher-level validator-selection or staking system can act on that evidence to impose penalties.

報告上鏈與資料可用

Reporting and data availability

Q041 §11 §15

一份 report 有足夠的 guarantor 簽名。區塊作者為什麼還要看它對應的 core 是否空著？

Why must a block author check that a core is free even when a report has enough guarantor signatures?

> 每個 core 同時只保留一個待確認的 availability assignment。前一份指派尚未清除時，新的 guarantee 不能占用它。
>
> Each core has room for one outstanding availability assignment. A new guarantee can take the core only after the previous assignment has cleared.

Q042 §11 §16

「已有人擔保這份 report」和「它已 available」分別需要什麼證據？

What evidence distinguishes a guaranteed report from an available report?

> Guarantee 由該 core 指派的 guarantors 簽署。成為 available 則需在區塊的 assurances 中，超過三分之二的 active validators 標記該 core 的資料已取得。
>
> A guarantee carries signatures from assigned guarantors. Availability requires assurances marking that core from more than two-thirds of active validators in the block.

Q043 §11

可用性聲明只列出哪些 core 的資料已收到。為何簽名還要綁定 parent hash？

Why bind an assurance to the parent hash when its bitfield already names the cores?

> 同一個 core 在不同鏈背景可能掛著不同的報告。簽入 parent hash 能讓節點把聲明對準正確的指派，並拒絕跨背景重放。
>
> A core can refer to different reports in different chain contexts. Signing the parent hash ties the bitfield to the intended assignments and prevents replay across those contexts.

Q044 §11 §15

Guarantors 簽完後剛好輪替，報告送到區塊作者時該用哪一組指派來驗簽？

Guarantors rotate just after signing. Which assignment should a block author use to verify their report?

> 依 guarantee 的 timeslot 找出當時該 core 的指派，再套用允許的輪替窗口。符合條件的上一輪簽名也能被核對。
>
> Use the assignment for the guarantee's timeslot and enforce the admitted rotation window. Signatures from the immediately preceding rotation may still qualify.

Q045 §11

某個 core 的報告一直收不到足夠的 assurances。協定怎麼讓 core 能繼續接工作？

How can a core return to service if its pending report never receives enough assurances?

> Pending assignment 到達 timeout 會被清空，core 可以再接新 guarantee。過期這件事本身不會讓舊報告變成 available。
>
> The pending assignment is cleared at timeout, freeing the core for a new guarantee. Expiry alone does not establish availability for the old report.

計算結果何時動到帳本

When computation changes the ledger

Q046 §12 §14

B 的成交結果依賴 A 的清算，但 B 先變成 available。為什麼不能只按資料到達順序寫入帳本？

Settlement B depends on A but becomes available first. Why would arrival order be unsafe for the ledger?

> B 需要 A 的效果先成立。協定保留未滿足的套件依賴，利用已完成的歷史和本區塊的依賴排序，把 A 排到 B 前面。
>
> B requires A's effects first. JAM tracks outstanding package dependencies and uses accumulated history plus within-block dependency ordering to place A before B.

Q047 §12

依賴排序已把 A 和 B 都列進候選，但本區塊只處理了 A。哪一個能記進完成歷史？

Dependency ordering selects both A and B as candidates, but the block processes only A. Which one enters accumulated history?

> 只有 A。候選順序讓節點知道可依什麼次序執行，寫入完成歷史的是實際處理的 report 前綴所對應的套件。
>
> Only A. Candidate order establishes an execution order. History records the packages in the prefix of reports actually processed.

Q048 §12

排在前面的 report 很貴，後面的很便宜。節點可以自行跳過前者來提高 gas 利用率嗎？

Can a node skip an expensive report at the front and process a cheaper one behind it?

> 各節點必須從同一序列選出預算容許的最長前綴。自行跳過某筆會改變執行集合，也可能改變後續依賴所看到的狀態。
>
> Nodes take the longest affordable prefix of the prescribed sequence. Skipping a report changes the executed set and may change the state observed by dependent work.

Q049 §12

同一區塊有多份 reports、也有多個 services。協定怎麼分配 gas、平行執行，又合併成一個結果？

How does a block budget several reports, execute services in parallel and merge their results?

> 外層選本輪可負擔的 reports。同輪各服務從相同的起始狀態處理自己的輸入，協定合併結果後，下一輪才接續。
>
> The outer step selects an affordable batch. Services process their own inputs from the same round state. The protocol merges their results before the next round begins.

Q050 §12

這個區塊沒有任何 work-item 指向服務 S。它還可能因哪些事情執行 accumulate？

What can trigger accumulation for service S when no work-item targets it in this block?

> 它可能收到 deferred transfer，或本區塊起始的 always-accumulate 配置給它基本 gas。後者在第一輪帶入。
>
> S can receive a deferred transfer or be listed for a base gas allowance under always-accumulate. That allowance is supplied in the block's first round.

同一區塊裡的輪次與紀錄

Rounds and accounting within a block

Q051 §12

DEX 在 accumulation 中送出 deferred transfer。收款服務最早什麼時候可以處理它？

A DEX emits a deferred transfer during accumulation. When can the receiving service process it?

> 本輪先完成各服務的執行與合併，轉帳再成為同一區塊下一個 accumulation round 的輸入。接收方可用轉帳攜帶的 gas 處理它。
>
> The current round finishes and merges first. The transfer becomes an input to the next round in the same block, carrying a gas allowance for its recipient.

Q052 §9 §12

Manager 在本輪把某個 core 的 assigner 從 A 換成 B。本輪採用誰輸出的 queue，B 何時能行使新角色？

The manager replaces core assigner A with B in this round. Whose queue is merged now, and when can B exercise the role?

> 本輪用起始時持有角色的 A 所輸出的 queue。角色變更合併進新狀態後，後續輪次才會把 B 當作該 core 的 assigner。
>
> This round takes the queue from A, the holder at its start. Following rounds see B as the assigner after the role update is merged.

Q053 §13

想比較驗證者一段時間的貢獻，又想分析某個區塊的服務負載，為何要看不同時間尺度的統計？

Why use different time windows to compare validator contributions and diagnose one block's service load?

> Validator activity 按 epoch 累積，保留當期與前一期紀錄。Core 和 service 的統計則描述單一區塊。
>
> Validator activity accumulates over an epoch, with current and previous records retained. Core and service statistics describe individual blocks.

Q054 §10 §13

如果 JAM 沒有直接替每次工作發獎勵，為什麼鏈上仍要記錄驗證者活動？

Why record validator activity on-chain if JAM does not pay each reward directly?

> 出塊、擔保與 assurance 等可觀察活動，能形成各節點一致核對的紀錄。上層 staking 系統可利用這些紀錄決定獎勵。
>
> Block production, guarantees and assurances leave objective evidence in the chain. A higher-level staking system can use those records when allocating rewards.

Q055 §12 §13

某服務在這個區塊有 refinement gas 紀錄，accumulation 的工作量卻是零。怎麼解釋？

How can a service show refinement gas in a block but no accumulation work?

> Refinement 統計來自本區塊新報告的工作。Accumulation 統計看這個區塊實際處理的項目與轉帳，同一份工作的兩個階段可能落在不同區塊。
>
> Newly reported work contributes refinement statistics. Accumulation statistics follow the items and transfers actually processed, which can occur in a later block.

Work-package 裡究竟裝什麼

What a work-package carries

Q056 §4 §14

一個 work-item 能不能放一百筆 DEX 訂單？JAM 怎麼知道這些位元組的意思？

Can one work-item carry a hundred DEX orders, and who decides what those bytes mean?

> 可以由服務把一批訂單編進 payload，只要符合資源限制。Work-item 指定一個服務，該服務的 refine 程式解讀 payload。
>
> A service can encode a batch of orders in a payload within the resource limits. A work-item targets one service, whose refinement code interprets those bytes.

Q057 §11 §12 §14

一個被成功評估的 work-package 裡有三個 items，分別給兩個服務。會產生幾份 report，結果怎麼分送？

A successfully evaluated package contains three items targeting two services. How many reports result, and how are the outputs routed?

> 對這次評估產生一份 work-report，內含每個 item 各自的 digest。累積時，這些結果依目標 service 分組送入執行。
>
> That evaluation yields one report containing a digest per item. Accumulation groups the relevant results by their target service.

Q058 §11 §14

某個 item 的 refine 耗盡 gas。這會怎麼反映在報告裡，審核報告正確性時該比對什麼？

An item runs out of gas during refinement. How does that affect its digest and the check on report correctness?

> 該 digest 可以記錄 out-of-gas 這類工作錯誤。審核時比對重跑的結果是否與報告一致，正確記錄的執行錯誤也是可驗證的結果。
>
> The digest can record an out-of-gas work error. An auditor checks that reproduction yields the reported outcome, including a correctly reported execution failure.

Q059 §12 §14

為什麼同一個 work-item 要分別聲明 refinement 和 accumulation 的 gas limit？

Why give one work-item separate gas limits for refinement and accumulation?

> 兩個階段使用不同執行環境。前者限制 core 上的計算，後者讓節點安排之後更新共享狀態所需的預算。
>
> The stages run in different execution contexts. One limit bounds in-core computation, while the other budgets the later on-chain state update.

Q060 §14 §16

服務 B 只需要服務 A 先前產出的一段大型資料，下一份工作怎麼取得並驗證它？

How can service B reuse and verify one piece of a large output previously produced by service A?

> A 的工作可匯出 segments，由 D^3L 分散保存，work-report 承諾其 segments root。B 的工作引用片段，取回後用證明對照該 root。
>
> A's work exports segments to D^3L and commits to their root in its report. B's work references the segment, retrieves it and checks its inclusion proof.

Guarantor 實際承諾什麼

What a guarantor commits to

Q061 §14 §15

Guarantor 的簽名交給區塊作者後，其他人能從這個簽名追究哪一項承諾？

What claim can others hold a guarantor accountable for after its signature reaches a block author?

> 簽名綁定具體 work-report 的雜湊，表示這名驗證者擔保該報告忠實反映規定的計算。日後可重算並核對這個承諾。
>
> The signature binds the guarantor to a particular report hash and its claimed computation. Others can reproduce the work and hold the signer to that claim.

Q062 §11 §15

某個 active core 的一名 guarantor 暫時離線，另外兩位算出相同的報告。這份工作還有機會上鏈嗎？

One guarantor for an active core is offline. Can the other two still produce an includable guarantee if they agree?

> 每個 active core 指派三位驗證者，兩位指派成員的有效簽名就可滿足簽名門檻。報告仍須通過其他納入條件。
>
> Three validators are assigned per active core. Two valid assigned signatures can meet the signing threshold, provided the other inclusion rules also hold.

Q063 §8 §11 §15

Guarantor 準備花很多算力跑 refine。有哪些便宜的檢查值得先做，為什麼？

Which cheap checks should a guarantor make before an expensive refinement, and why?

> 先檢查授權、對應 core 的 pool 與報告背景是否可能符合納入規則。即使計算結果正確，不合納入條件也可能白做。
>
> Check authorization, core-pool membership and contextual eligibility first. An accurately computed report can still be ineligible for inclusion.

Q064 §11 §15

同一個 core 的 guarantors 各自拿到不同套件。如果互不協調，可能浪費什麼工作？

What work is wasted if guarantors on the same core independently choose different packages?

> 同一個 guarantee 需要對同一份 report 的簽名。提早分享候選套件能讓同 core 的成員評估同一份工作，再彙集相符的簽名。
>
> A guarantee needs signatures on the same report. Sharing a candidate package early helps guarantors evaluate the same work and combine matching attestations.

Q065 §14 §15 §16

Guarantor 已經把 report 送給區塊作者。為什麼它還要向驗證者網路分發資料？

After giving the report to a block author, why must guarantors also distribute data to the validator network?

> 其他驗證者需要資料來稽核，後續工作也可能要匯入輸出。Guarantors 因此把可稽核的 bundle 與匯出 segments 做糾刪編碼並分發分片。
>
> Auditors need the inputs, and later work may need the exports. Guarantors erasure-code and distribute both the auditable bundle and the exported segments.

資料能不能取回

Can the data be recovered?

Q066 §16 App. H

驗證者準備對一個 core 簽 assurance。它在簽名前要親自取得並核對什麼？

What must a validator obtain and check before signing an assurance for a core?

> 它要持有分配給自己的 bundle 分片及相應的匯出片段分片，並核對對應位置的證明，才能聲明持有資料。
>
> It must hold its assigned bundle shard and the corresponding shards of exported segments, and verify the proofs for their positions before claiming possession.

Q067 §14 §16

對同一份 work-package，稽核用的 bundle 與留給後續工作的匯出資料，為什麼有不同保存需求？

Why do the auditable bundle and a package's exported data have different retention needs?

> Bundle 服務近期的重跑稽核。匯出資料屬於較長期的 D³L，其他工作日後仍可匯入，規格要求把相關分片保留至少 28 天。
>
> The bundle serves near-term auditing. Exports go into the longer-lived D³L so later work can import them. Their shards are retained for at least 28 days.

Q068 §14 §16

舊工作匯出的資料還在保存期內，但驗證者已經換屆。要向哪一批人請求分片？

An export is still within its retention window, but validators have rotated. Which set should a node fetch shards from?

> 依當時對該資料作 assurance 的驗證者集合與分片位置取得資料。現在的 active set 可能已經換人，甚至規模也可能不同。
>
> Use the validator set that assured the data and its shard indices. Membership and set size may both have changed since then.

Q069 §14 §16 App. H

一個有 Merkle 證明的分片，和足夠多個能還原資料的分片，分別解決哪個問題？

What does proving one shard establish, and what does collecting enough distinct shards establish?

> 證明用來驗證分片確實位於承諾的位置。收集達門檻的不同有效分片，糾刪解碼才可還原整份資料。
>
> The proof authenticates the shard at its committed position. A sufficient set of distinct valid shards lets the erasure decoder recover the full data.

Q070 §11 §16 App. H

一份報告需要超過三分之二的 availability assurances，但重建時只取部分分片。這兩個門檻各保護什麼？

Why can availability require a validator supermajority even though recovery uses only a subset of shards?

> 聲明門檻是協定認定資料已分散持有的條件。重建門檻是編碼的數學性質，決定需要多少個不同的有效分片才能解碼。
>
> The assurance threshold is the protocol's evidence of distribution. The recovery threshold is a property of the code that determines how many distinct valid shards suffice to decode.

為什麼抽查能建立信任

Why sampled audits work

Q071 §11 §17

一份 report 已經 available。Auditor 親自重跑時，能多回答哪個安全問題？

Once a report is available, what additional security question can an auditor answer by re-executing it?

> 可用性讓它能取得輸入。Auditor 重建並核對 bundle，重算報告，檢查 guarantors 提交的內容是否忠實反映計算。
>
> Availability makes the inputs retrievable. The auditor verifies the bundle and recomputes the report to check whether the guarantors reported the computation faithfully.

Q072 §17 §20 App. G

每份工作平常只有少數人稽核。攻擊者為什麼很難先挑一批可以買通的人，讓無效結果過關？

If only a sample normally audits each package, why is it hard to preselect and bribe everyone who will check it?

> 稽核分工由驗證者各自用 VRF 抽選，事前難以預測，公告後可驗證。抽到誠實稽核者就可能發現錯誤並啟動更廣泛檢查。
>
> Validators use private VRF-based selection and later prove their assignments. An attacker has difficulty predicting every checker, and an honest negative judgment escalates checking.

Q073 §17 §20

全規模設定下，每位驗證者初始抽十個 core。為什麼一份 report 平均約有三十次稽核？

At full scale, each validator initially samples ten cores. Why does a report receive about thirty audits on average?

> 兩個數字的分母不同。按 1,023 位驗證者、341 個 active cores 的模型，平均每個 core 被選中的次數是 $1{,}023 \times 10 \div 341$，也就是 30。
>
> One figure is per validator and the other per report. In the 1,023-validator, 341-active-core model, 1,023 times 10 divided by 341 gives 30 expected selections per core.

Q074 §10 §17

已宣布的稽核遲遲沒有回覆，和有人明確發出負面 judgment，接下來的處理有何不同？

How does a missing announced judgment differ from an explicit negative judgment in subsequent auditing?

> 缺席會讓後續 tranche 依 VRF 規則選出額外稽核工作。看到負面 judgment 時，所有驗證者都應就該報告稽核並提出判斷。
>
> A no-show can assign additional checks through VRF rules in later tranches. A negative judgment requires all validators to audit that report and issue a judgment.

Q075 §4 §14 §17

預測市場使用外部比分資料。計算結果達到 finality 之後，使用者能信任哪一段，比分真實性又依賴誰？

A prediction market reads an external score. After finality, what can users trust, and who vouches for the real-world score?

> 在協定安全假設下，可高度信任程式對指定輸入的計算與鏈上結果。比分是否真實，仍取決於應用採用的資料來源與驗證規則。
>
> Under the protocol assumptions, users can trust the computation on the specified input and its finalized effects. The real-world score depends on the application's data source and validation rules.

最終性與跨鏈證明

Finality and external proofs

Q076 §18 §19

另一條鏈想驗證 JAM 的最終結果，卻不想重播整條鏈。BEEFY 在這裡提供什麼？

A bridge wants evidence of a finalized JAM result without replaying the whole chain. What does BEEFY provide?

> 驗證者匯入 finalized block 後，對對應的 log 承諾作 BLS 簽名。簽名可以聚合，讓外部系統取得精簡、可驗證的最終性證據。
>
> After importing finalized blocks, validators sign the relevant log commitment with BLS keys. Aggregation yields compact finality evidence for an external verifier.

Q077 §7 §18 App. E

橋接端已驗證一個 BEEFY 承諾，還想確認某個服務發布的個別輸出。兩份證據要怎麼接起來？

A bridge has verified a BEEFY commitment. How does it connect that commitment to one service output?

> 先證明服務 ID 與輸出 hash 收在相應區塊的輸出根，再證明該根屬於已簽署承諾覆蓋的輸出歷史。
>
> Prove that the service ID and output hash belong to the block's output root. Then prove that root is included in the output history covered by the signed commitment.

Q078 §6 §17 §19

節點看到兩個競爭鏈頭。比較哪條鏈比較好之前，要先排除哪些候選？

Before comparing two chain heads, which candidates must a node rule out?

> 候選須延續已 finalized 的區塊，通過本地的 audited 條件，且未確定的路徑不可包含已觀察到的同 slot 雙重出塊。
>
> A candidate must descend from the finalized block, satisfy the local audit condition and have no observed equivocation in its unfinalized ancestry.

Q079 §6 §19

兩條分支都符合候選條件，較長的一條卻有許多 fallback blocks。最佳鏈怎麼選？

Both forks are acceptable, but the longer one has many fallback blocks. How is the best chain chosen?

> 比較候選歷史中以一般 slot-sealer ticket 封成的區塊數，選使該數量最大的鏈頭。區塊總數本身不足以決定結果。
>
> Among acceptable heads, maximize the number of ticket-sealed blocks in the ancestry. A longer fork with more fallback blocks need not win.

Q080 §5 §19

兩個驗證者投給相同的 header，算出的 post-state root 卻不同。GRANDPA 會把票算在一起嗎？

Two validators vote for the same header but attach different post-state roots. Do those votes support the same target?

> JAM 的 GRANDPA 投票同時承諾 header 和執行後的狀態根。根不同就屬於不同投票目標，這會暴露執行結果的歧異。
>
> A JAM GRANDPA vote binds the header together with its post-state root. Different roots make different voting targets and expose disagreement about execution.

效能、設計取捨與範圍

Performance and design trade-offs

Q081 §20

有人用理論 TPS 比較 JAM 和其他鏈。要把這個數字用到你的 DEX，至少要先問哪些工作負載假設？

Before applying a theoretical JAM TPS estimate to a DEX, which workload assumptions would you check?

> 問每筆工作要搬多少資料、讀寫多少共享狀態、能否切成獨立批次，以及硬體與網路預算。吞吐量要用符合這些條件的實際負載驗證。
>
> Check bytes moved, shared-state accesses, available parallelism and hardware and network budgets. Benchmark a representative workload before using the estimate.

Q082 §20 §21

有人主張未來能把 Polkadot parachains 放進 JAM。這個設計主張要靠什麼實驗才能估算實際吞吐量？

What evidence would turn a design for hosting parachains on JAM into a credible throughput estimate?

> 以服務原型實際執行 parachain 驗證工作，量測 PVM 計量、資料存取與整合的成本，再和模型的資源假設比較。
>
> Prototype the parachain-hosting service with representative validation workloads. Measure metered PVM execution, data access and integration costs against the model's assumptions.

Q083 §4 §13 §21

團隊打算實作 coretime 銷售與 staking 獎勵。Gray Paper 已提供哪些底層能力，哪些政策要另行定義？

Which protocol capabilities can a coretime market or staking system use, and which policies must it define itself?

> 底層定義 core 授權、驗證者集合的銜接與鏈上活動紀錄。如何定價出售 coretime、如何按貢獻獎懲，交給較高層的系統。
>
> The protocol supplies core authorization, validator-set transitions and activity records. A higher-level system defines coretime pricing, sales and staking incentives.

Q084 §2 §6 §17 §22

JAM 沿用的 ELVES 與 Sassafras 思路，各在保護哪一件事？

What distinct problems do the ELVES and Sassafras ideas inherited by JAM address?

> ELVES 的擔保、抽查與爭議機制支持平行工作的安全性。Safrole 借用並簡化 Sassafras 的思路，安排誰有資格出塊。
>
> ELVES supplies the security pattern for backing, auditing and disputing parallel work. Safrole adapts Sassafras ideas to determine who may author blocks.

Q085 §12 §21

設計跨服務交易時，怎麼判斷「同步呼叫另一個服務」是否是 v0.8.0 已能依賴的行為？

Can a v0.8.0 service design rely on synchronous calls to another service during accumulation?

> 應照現行 invocation 與狀態轉移定義實作。跨服務 deferred transfers 已有規則，accumulate 中的同步呼叫則列在結論的後續研究。
>
> Use the specified invocation and transition rules. Deferred transfers have defined semantics. Synchronous inter-service calls during accumulation are listed as further work.

同一套 PVM，三種執行情境

One PVM, different invocation contexts

Q086 §4 App. A

同一支程式在慢機器上跑了兩倍久。兩個節點為什麼仍應在相同位置耗盡 gas？

The same program takes twice as long on a slower computer. Why should both nodes exhaust gas at the same point?

> 給定相同的機器與主機環境、輸入及起始 gas，兩者依規定的操作成本扣費。計費取決於協定步驟與執行狀態，而非牆上時鐘。
>
> With identical machine and host contexts, input and initial gas, both apply the protocol's cost rules. Metering follows execution steps and state independently of wall-clock speed.

Q087 App. A

一個陌生服務嘗試寫入沒有寫入權限的記憶體。為什麼這也屬於共識規格？

Why must consensus specify what happens when unknown service code writes to an inaccessible page?

> PVM 定義自己的記憶體、頁面權限及相應的 fault 行為。獨立實作要觀察到同樣的機器結果，才可一致處理這次 invocation。
>
> The PVM defines its memory, page permissions and resulting fault behavior. Independent clients must observe the same machine outcome to handle the invocation consistently.

Q088 §9 App. A App. B

Refine 和 accumulate 跑在同一套 PVM 上。為什麼服務在兩個入口能對鏈上狀態做的事不同？

Why do refine and accumulate have different powers over chain state if both run on the PVM?

> 呼叫情境決定可用的 host calls 與資料視角。Refine 取得受限的歷史輸入，accumulate 的環境則提供鏈上狀態變更能力。
>
> Each invocation defines its host calls and data view. Refinement has restricted historical inputs, while accumulation receives capabilities to change on-chain state.

Q089 App. B

資料很大的時候，為什麼 invocation 讓服務按需取資料，而不在啟動時把所有輸入一次塞進 RAM？

Why let service code fetch large inputs on demand instead of loading everything into RAM at startup?

> 這樣可避免不受程式控制的大量初始配置與 gas 成本。服務透過對應 host call 讀取實際需要的資料範圍。
>
> It bounds initial allocation and avoids large gas costs before the program can act. The service can request the data ranges it actually needs through host calls.

Q090 §12 App. B

服務在 accumulate 中先做部分更新，再耗盡 gas。有 checkpoint 和沒有 checkpoint，會怎樣影響保留的效果？

How can a checkpoint affect what survives if an accumulation invocation later runs out of gas?

> 異常結束時會選用 exceptional context。Checkpoint 可把當時的效果存入該 context，之後尚未 checkpoint 的變更不會因此一併保留。
>
> An exceptional exit selects the exceptional context. A checkpoint can save effects there, while later changes that were not checkpointed do not automatically survive.

把狀態與資料變成可驗證的承諾

Encoding and Merkle commitments

Q091 App. C

兩個團隊對同一個 header 的欄位值完全同意，卻算出不同的 hash。除了 hash 函式，還要查哪裡？

Two teams agree on every header field but compute different hashes. Besides the hash function, what should they compare?

> 先比對送進雜湊函式的完整位元組。協定 codec 的欄位順序、整數表示與長度編碼都會影響承諾。
>
> Compare the exact bytes passed to the hash function. Field order, integer representation and length encoding in the protocol codec all affect the commitment.

Q092 §3 App. C

兩段可變長度資料接在一起，接收方靠什麼找出分界並還原原值？

How can a decoder recover field boundaries when two variable-length values are encoded next to each other?

> 依雙方共用的型別結構和 codec 規則，對需要長度資訊的欄位使用規定的長度前綴。固定長度欄位則由其型別決定邊界。
>
> The shared type structure and codec specify where lengths must be encoded. Length prefixes delimit variable fields where required, while fixed-size types provide their own boundaries.

Q093 App. D

兩台節點把服務存在不同資料庫、資料列插入順序也不同。它們如何仍能得到同一個 state root？

How can clients with different databases and insertion orders still obtain the same state root?

> 先依協定把狀態映成標準的 state keys 與值，再按同一棵 Merkle trie 的規則取根。相同邏輯狀態便有相同承諾。
>
> Both map the logical state to the specified keys and values, then apply the same Merkle trie construction. The commitment is determined by that mapping.

Q094 §7 App. D App. E

輕節點要查「某個 storage key 的值」和「某筆服務輸出有沒有出現在清單裡」，該分別對哪種根驗證？

Which commitment should a light client use for a storage value versus membership in a service-output list?

> 儲存值依 state key 對 state-trie root 驗證。清單成員則依位置與包含證明對輸出序列的 Merkle root 驗證。
>
> Verify a storage value at its state key against the state trie root. Verify list membership at its position against the output sequence's Merkle root.

Q095 §7 App. E

每個區塊都追加一個輸出根，為什麼用 Merkle mountain 結構適合維護這段歷史？

Why use a Merkle mountain structure for a history that gains an output root each block?

> 追加時可重用既有子樹，按需要合併 peaks，得到新的承諾。節點能持續延長歷史，也能替過去的項目提供證明。
>
> Appending reuses existing subtrees and merges peaks as needed to update the commitment. The history can grow while retaining support for proofs about earlier entries.

把各章接成一個可驗證的系統

Putting the protocol together

Q096 §6 §11 App. F

驗證者指派需要洗牌。如果每個節點各用自己的隨機產生器洗一次，其他人還怎麼驗證指派？

How can every node verify a shuffled assignment instead of drawing its own unrelated permutation?

> 協定指定相同的輸入順序、seed 和洗牌演算法。各自重算會得到同一個排列，因此同一份簽名能對上同一個指派。
>
> Nodes use the specified input order, seed and shuffle algorithm. They reconstruct the same permutation and can check a signature against the same assignment.

Q097 §6 §15 §18 App. G

簽署一份 guarantee、證明匿名出塊票券、簽署 BEEFY 承諾，為什麼需要三種不同的驗證方式？

Why verify a guarantee, an anonymous ticket and a BEEFY commitment in different ways?

> Guarantee 要辨認為 report 負責的簽署者。Ring VRF 證明票券來自合格集合並產生可驗證輸出，BEEFY 的 BLS 簽名則方便聚合供外部驗證。
>
> Guarantees identify signers responsible for a report. Ring VRFs prove eligible membership and a verifiable output for tickets. BLS signatures let external verifiers check aggregated BEEFY commitments.

Q098 §16 App. H

如果改成每位驗證者都留完整 bundle，和現行把糾刪碼分片分散保存相比，取捨在哪？

What trade-off changes if every validator stores a full bundle instead of an erasure-coded shard?

> 完整複本方便單點讀取，代價是反覆傳送與保存整份資料。分片把負擔分散，讀取方需取得足夠的不同有效分片再重建。
>
> Full replicas allow direct local reads but duplicate storage and transmission. Sharding spreads that cost, while recovery requires enough distinct valid shards.

Q099 §3 §6 App. I

兩個實作用了相同公式，卻把 epoch 長度設成不同值。為什麼可能演變成共識分歧？要去哪核對？

How could different epoch-length settings cause consensus divergence even with identical formulas, and where would you check the value?

> 同一個 slot 可能被放進不同 epoch，造成輪替時點或有效金鑰不同。用該版 Appendix I 的常數索引及對應章節核對網路參數。
>
> The same slot may land in different epochs, changing rotation timing and valid keys. Check the version's constants in Appendix I and their definitions in the relevant section.

Q100 §4 §7 §12 §19 App. D

使用者說「這筆 DEX 訂單已成交，結果也已確定」。你會怎麼從工作狀態一路驗到這筆具體成交？

How would you verify the claim that a particular DEX order both filled and reached finality?

> 沿套件、report、可用性與 accumulation 查處理進度。最後用已 finalized 的狀態或輸出承諾，核對 DEX 實際記錄了該筆成交。
>
> Trace the package through reporting, availability and accumulation. Then verify the DEX-specific fill against a finalized state or output commitment.